

**Pandas****Seaborn****Plotly****Altair**

Introduction aux Diagrammes à Barres

Les diagrammes à barres sont un outil de visualisation fondamental utilisé pour afficher des données catégorielles avec des barres rectangulaires. La longueur de chaque barre est proportionnelle à la valeur qu'elle représente. Les diagrammes à barres sont excellents pour comparer des quantités entre différentes catégories.

Quand utiliser les diagrammes à barres :

- Comparaison de données catégorielles.
- Affichage de la fréquence ou du nombre de différentes catégories.
- Visualisation des tendances entre les catégories.

Terminologie de base :

- Barres : Les colonnes rectangulaires représentant les valeurs des données.
- Axes : Les lignes horizontales (axe des x) et verticales (axe des y) qui définissent le graphique.
- Étiquettes : Le texte qui décrit les catégories et les valeurs.
- Titre : La description générale du graphique.

Exercice : Analyse de Diamants avec les Bibliothèques de Visualisation

Avant de commencer cet exercice, veuillez lire la documentation des bibliothèques suivantes pour vous familiariser avec leurs fonctionnalités de base :

- Matplotlib : <https://matplotlib.org/>
- Seaborn : <https://seaborn.pydata.org/>
- Plotly : <https://plotly.com/python/>
- Altair : <https://altair-viz.github.io/>

Vous disposez d'un ensemble de données contenant des informations sur divers diamants. Votre tâche est de réaliser l'analyse et les visualisations suivantes en utilisant Matplotlib, Seaborn, Plotly et Altair. Pour chaque visualisation, fournissez un bref commentaire sur ce que vous observez concernant les forces et les faiblesses de chaque bibliothèque pour cette tâche particulière.

1. Chargement et Inspection des Données :

- Chargez l'ensemble de données à partir de l'URL fourni :
`"https://raw.githubusercontent.com/TrainingByPackt/Interactive-Data-Visualization-with-Python/master/datasets/diamonds.csv"`
- Affichez les 5 premières lignes de l'ensemble de données pour comprendre sa structure.
- Imprimez les noms des colonnes et leurs types de données.
- Affichez les 10 premières lignes de l'ensemble de données.

2. Visualisation par Diagramme à Barres :

- **Matplotlib :**
 - Créez un diagramme à barres pour visualiser le nombre de diamants pour chaque catégorie de 'cut' en utilisant Matplotlib.
 - Personnalisez ce diagramme à barres en ajoutant des étiquettes appropriées pour l'axe des x et l'axe des y, et définissez un titre pour le graphique.
 - Commentez la syntaxe de Matplotlib, ses capacités de personnalisation et sa facilité d'utilisation pour cette tâche.
- **Seaborn :**
 - Créez le même diagramme à barres (nombre de diamants par 'cut') en utilisant Seaborn.
 - Personnalisez le graphique comme vous l'avez fait avec Matplotlib (étiquettes, titre).
 - Comparez le code de Seaborn à celui de Matplotlib. Qu'est-ce que Seaborn fait différemment ou plus facilement ? Comment le style par défaut se compare-t-il ?
- **Plotly :**
 - Créez un diagramme à barres interactif (nombre de diamants par 'cut') en utilisant Plotly.
 - Personnalisez le graphique avec des étiquettes et un titre. Explorez les fonctionnalités d'interactivité de Plotly (info-bulles, zoom).
 - Commentez l'interactivité de Plotly, sa syntaxe et en quoi elle diffère de Matplotlib et Seaborn.
- **Altair :**
 - Créez un diagramme à barres (nombre de diamants par 'cut') en utilisant Altair.
 - Personnalisez le graphique avec des étiquettes et un titre.
 - Commentez la syntaxe déclarative d'Altair et comment elle se compare aux autres bibliothèques.

3. Analyse Supplémentaire (Facultatif) :

- Explorez davantage les données en créant des diagrammes à barres pour d'autres variables catégorielles telles que 'color' ou 'clarity' dans n'importe laquelle des bibliothèques.

- Essayez de visualiser le prix moyen pour chaque 'cut' ou 'color' en utilisant les différentes bibliothèques et commentez la façon dont chaque bibliothèque gère ce type d'agrégation.

Conseils :

- Utilisez la bibliothèque pandas pour charger et manipuler les données.
- Utilisez les bibliothèques Matplotlib, Seaborn, Plotly et Altair pour créer les diagrammes à barres.
- Référez-vous au notebook pour des exemples de code et des explications sur les fonctions dont vous pourriez avoir besoin.

```
In [6]: # Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
import altair as alt

# Désactive la limite de lignes pour les grands datasets.
alt.data_transformers.disable_max_rows()
```

```
Out[6]: DataTransformerRegistry.enable('default')
```

```
In [9]: # Chargement du dataset depuis l'URL
diamonds_url = "https://raw.githubusercontent.com/TrainingByPackt/Interactive-Da
diamonds_df = pd.read_csv(diamonds_url)
```

```
In [10]: # Affichage des 5 premières lignes
print("=== 5 premières lignes ===")
diamonds_df.head()
```

```
=== 5 premières lignes ===
```

```
Out[10]:
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75

```
In [11]: # Noms des colonnes et leurs types de données
print("=== Colonnes et types de données ===")
print(diamonds_df.dtypes)
```

```
=== Colonnes et types de données ===
carat      float64
cut         str
color       str
clarity     str
depth       float64
table       float64
price       int64
x           float64
y           float64
z           float64
dtype: object
```

```
In [12]: # Affichage des 10 premières lignes
print("=== 10 premières lignes ===")
diamonds_df.head(10)
```

```
=== 10 premières lignes ===
```

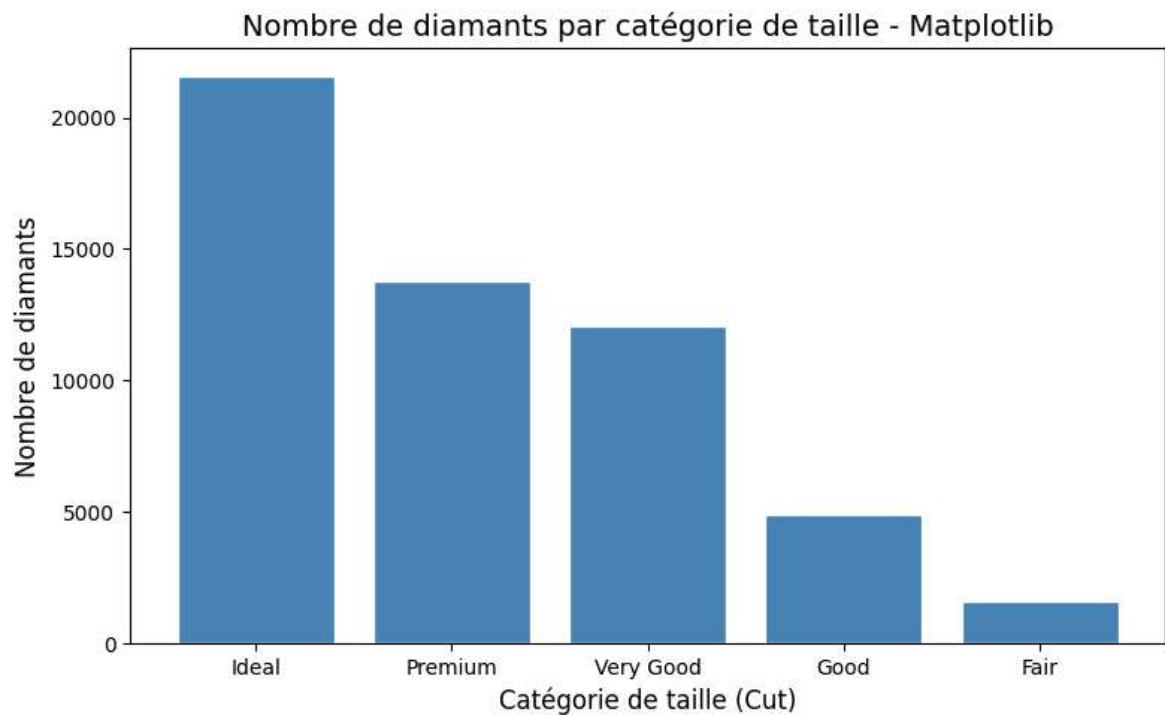
```
Out[12]:
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
5	0.24	Very Good	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
6	0.24	Very Good	I	VVS1	62.3	57.0	336	3.95	3.98	2.47
7	0.26	Very Good	H	SI1	61.9	55.0	337	4.07	4.11	2.53
8	0.22	Fair	E	VS2	65.1	61.0	337	3.87	3.78	2.49
9	0.23	Very Good	H	VS1	59.4	61.0	338	4.00	4.05	2.39

2. Visualisation par Diagramme à Barres - Nombre par Cut

```
In [13]: # Préparation des données : nombre de diamants par 'cut'
cut_counts = diamonds_df['cut'].value_counts().reset_index()
cut_counts.columns = ['cut', 'count']
```

```
In [14]: fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(cut_counts['cut'], cut_counts['count'], color='steelblue', edgecolor='whi
ax.set_xlabel('Catégorie de taille (Cut)', fontsize=12)
ax.set_ylabel('Nombre de diamants', fontsize=12)
ax.set_title('Nombre de diamants par catégorie de taille - Matplotlib', fontsize
plt.tight_layout()
plt.show()
# Matplotlib offre un contrôle précis sur chaque élément visuel,
# mais la syntaxe est verbeuse. Idéal pour des graphiques très personnalisés,
# mais moins adapté à une exploration rapide des données.
```

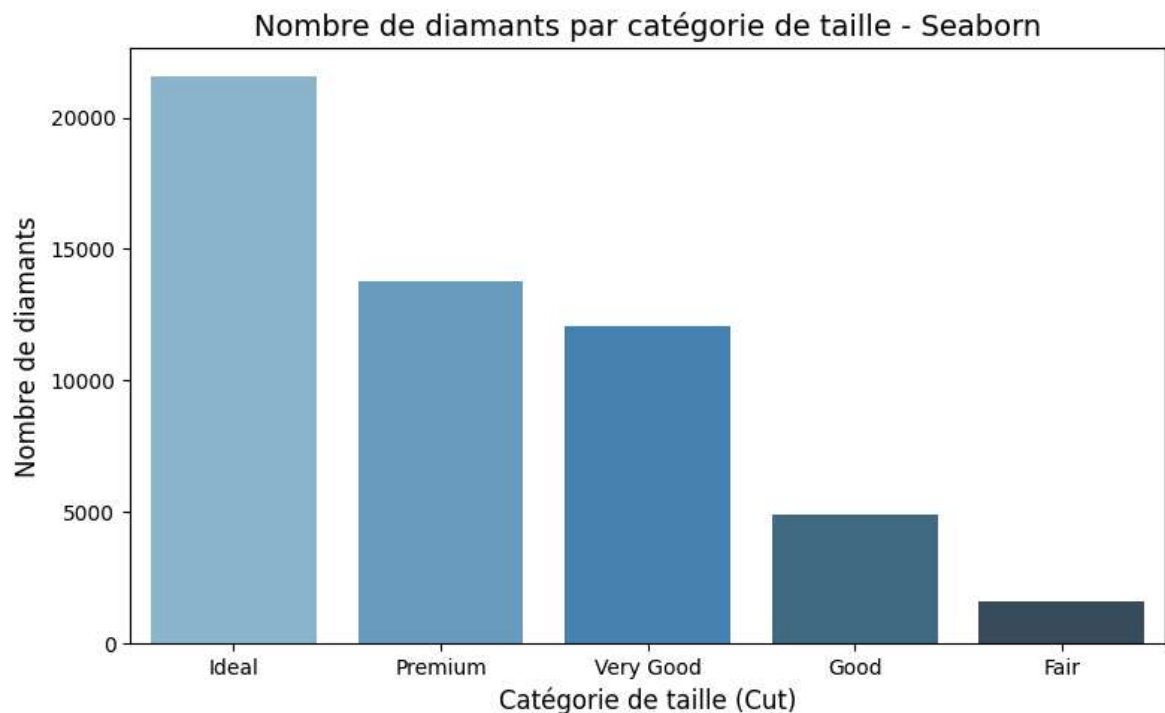


```
In [15]: plt.figure(figsize=(8, 5))
sns.countplot(data=diamonds_df, x='cut', order=cut_counts['cut'], palette='Blues')
plt.xlabel('Catégorie de taille (Cut)', fontsize=12)
plt.ylabel('Nombre de diamants', fontsize=12)
plt.title('Nombre de diamants par catégorie de taille - Seaborn', fontsize=14)
plt.tight_layout()
plt.show()
# Seaborn est plus concis : countplot() calcule automatiquement les effectifs
# sans préparer les données. Le style par défaut est plus soigné que Matplotlib.
# Cependant le graphique reste statique.
```

C:\Users\thoma\AppData\Local\Temp\ipykernel_123588\1812881335.py:2: FutureWarning:

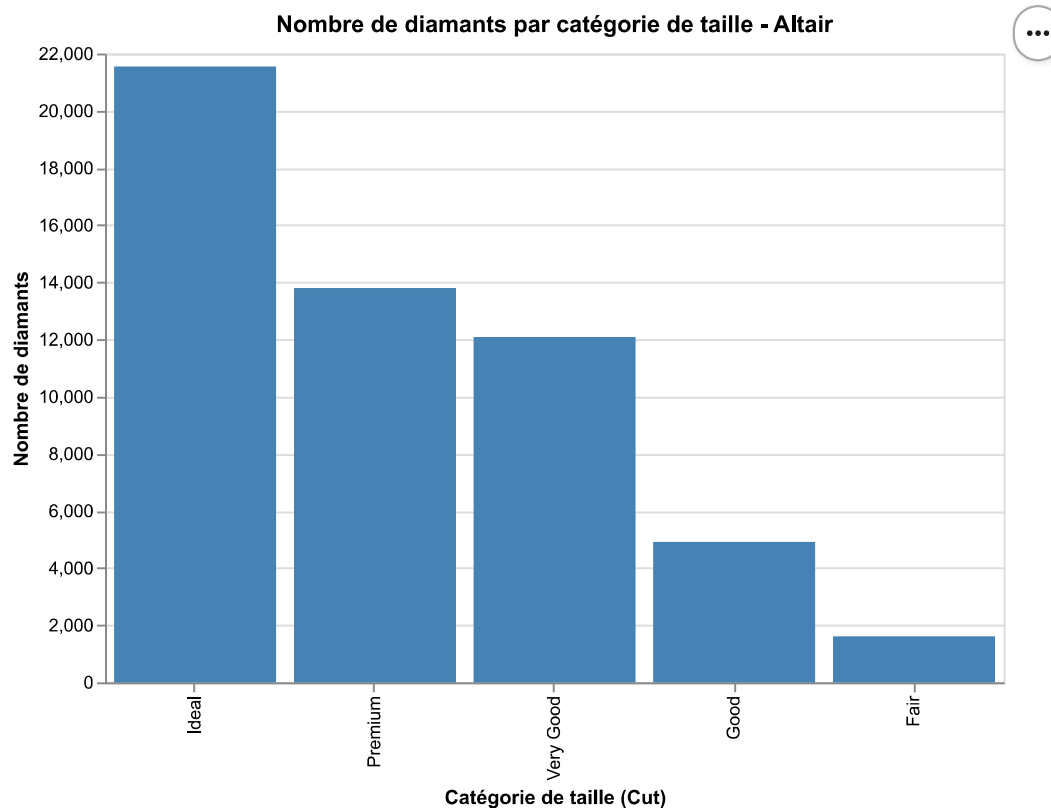
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data=diamonds_df, x='cut', order=cut_counts['cut'], palette='Blues_d')
```



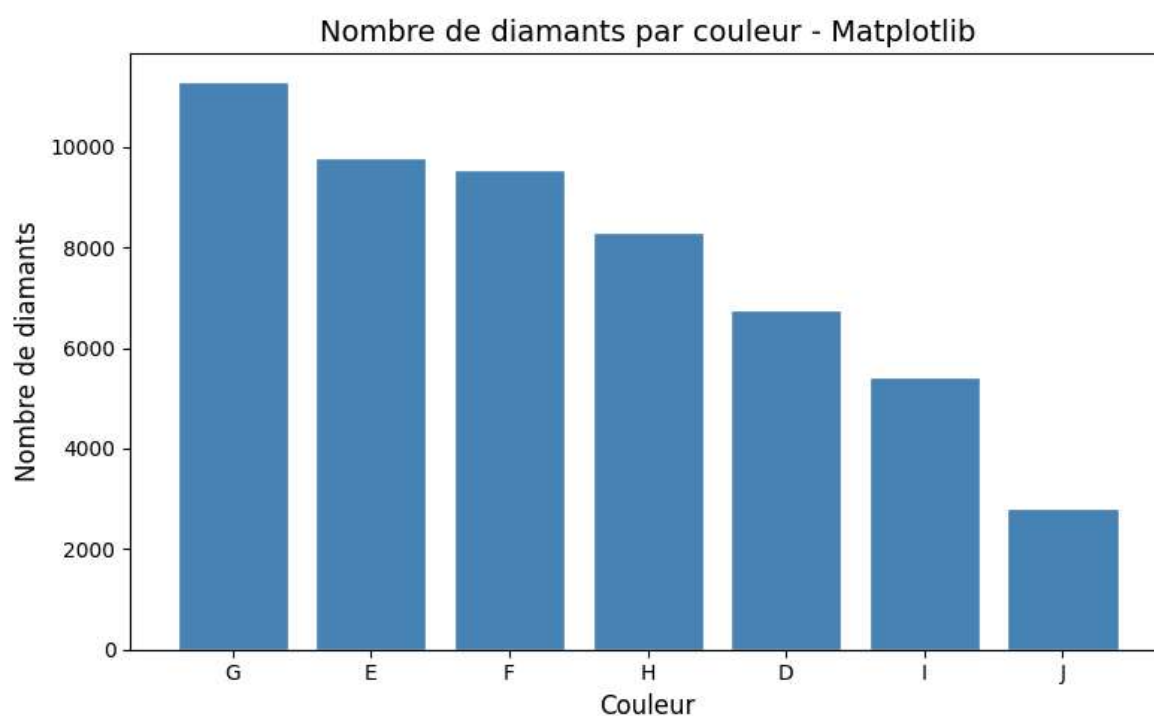
```
In [16]: fig = px.bar(
    cut_counts,
    x='cut', y='count',
    title='Nombre de diamants par catégorie de taille - Plotly',
    labels={'cut': 'Catégorie de taille (Cut)', 'count': 'Nombre de diamants'},
    color='count',
    color_continuous_scale='Blues'
)
fig.update_layout(coloraxis_showscale=False)
fig.show()
# Plotly génère des graphiques interactifs : zoom, tooltip au survol, export PNG
# Idéal pour des dashboards ou des notebooks interactifs.
```

```
In [17]: chart = alt.Chart(diamonds_df).mark_bar(color='steelblue').encode(
    x=alt.X('cut:N', title='Catégorie de taille (Cut)', sort='-y'),
    y=alt.Y('count():Q', title='Nombre de diamants'),
    tooltip=['cut:N', 'count():Q']
).properties(
    title='Nombre de diamants par catégorie de taille - Altair',
    width=500, height=350
)
chart.show()
# Altair utilise une syntaxe déclarative : on décrit CE QUE l'on veut visualiser
# plutôt que COMMENT le dessiner.
```



```
In [18]: color_counts = diamonds_df['color'].value_counts().reset_index()
color_counts.columns = ['color', 'count']
```

```
In [19]: # -- Matplotlib --
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(color_counts['color'], color_counts['count'], color='steelblue', edgecolor='black')
ax.set_xlabel('Couleur', fontsize=12)
ax.set_ylabel('Nombre de diamants', fontsize=12)
ax.set_title('Nombre de diamants par couleur - Matplotlib', fontsize=14)
plt.tight_layout()
plt.show()
```

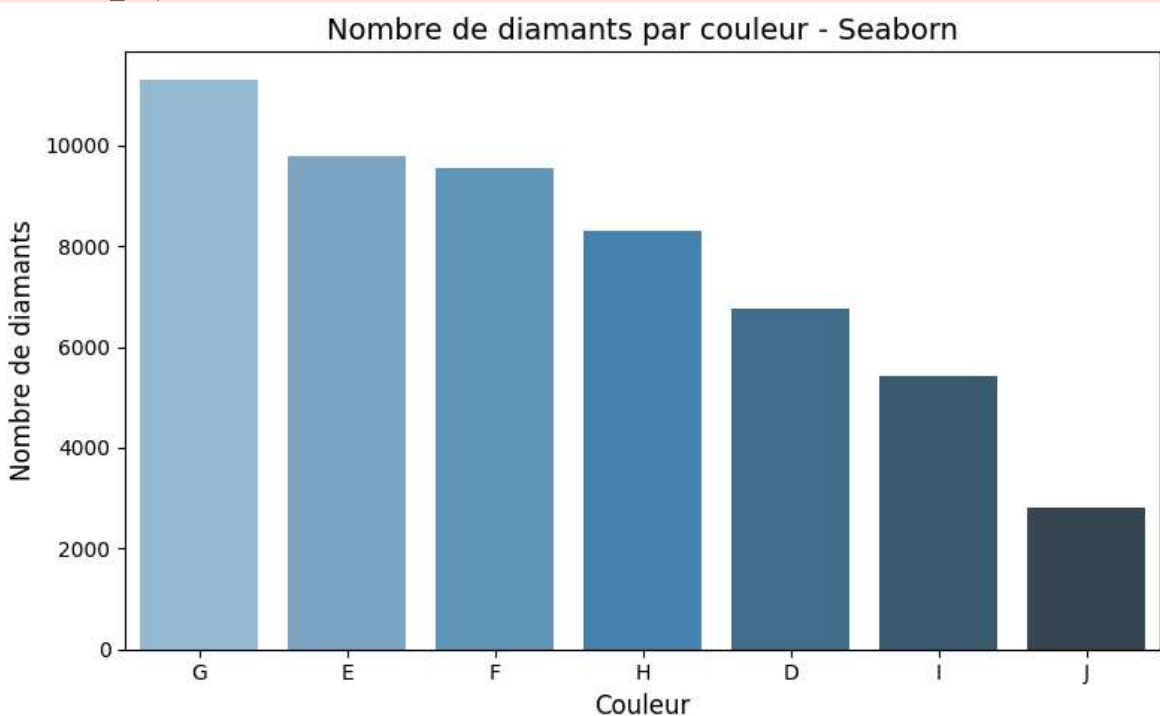


```
In [21]: # -- Seaborn --
plt.figure(figsize=(8, 5))
sns.countplot(data=diamonds_df, x='color', order=color_counts['color'], palette=
plt.xlabel('Couleur', fontsize=12)
plt.ylabel('Nombre de diamants', fontsize=12)
plt.title('Nombre de diamants par couleur - Seaborn', fontsize=14)
plt.tight_layout()
plt.show()
```

C:\Users\thoma\AppData\Local\Temp\ipykernel_123588\2859064011.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v 0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

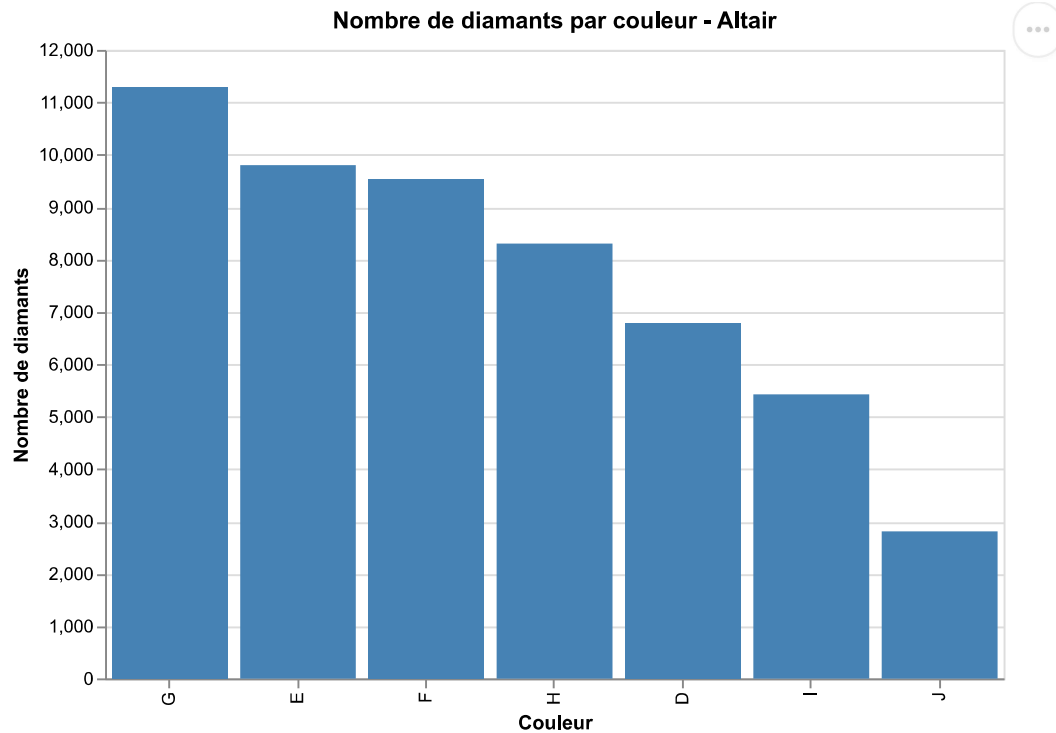
```
sns.countplot(data=diamonds_df, x='color', order=color_counts['color'], palette='Blues_d')
```



```
In [22]: # -- Plotly --
fig = px.bar(
    color_counts,
    x='color', y='count',
    title='Nombre de diamants par couleur - Plotly',
    labels={'color': 'Couleur', 'count': 'Nombre de diamants'},
    color='count', color_continuous_scale='Blues'
)
fig.update_layout(coloraxis_showscale=False)
fig.show()
```

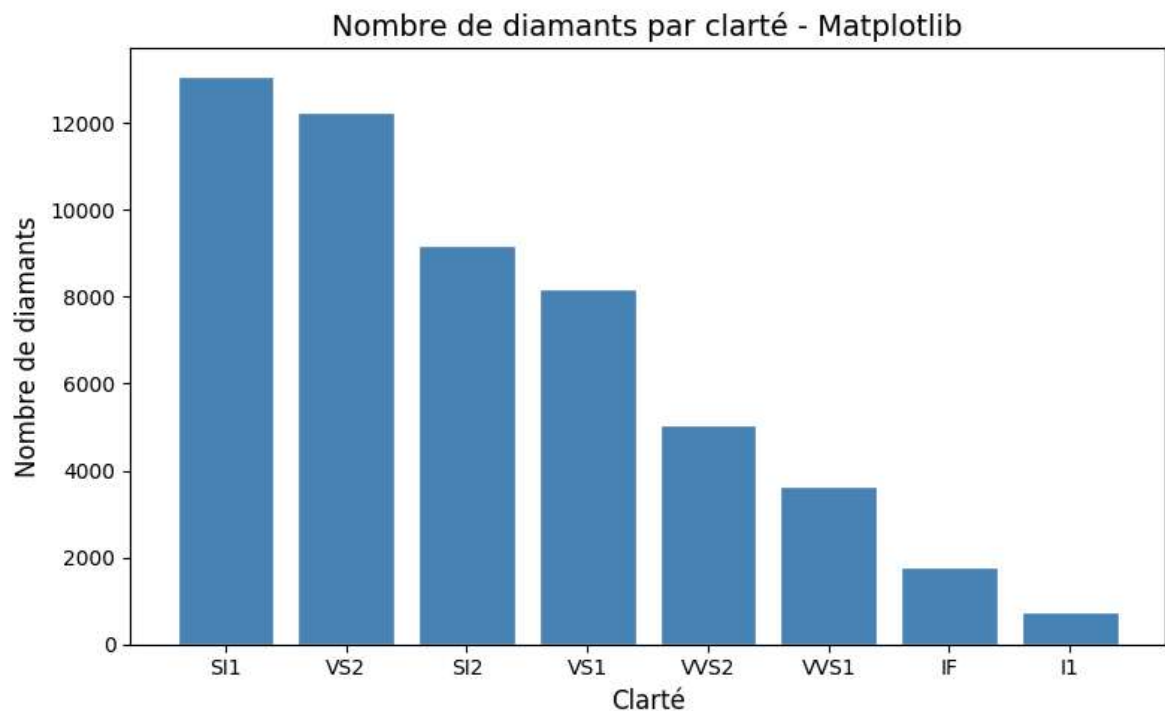
```
In [55]: # -- Altair --
chart = alt.Chart(diamonds_df).mark_bar(color='steelblue').encode(
    x=alt.X('color:N', title='Couleur', sort='-y'),
    y=alt.Y('count():Q', title='Nombre de diamants'),
    tooltip=['color:N', 'count():Q']
).properties(
    title='Nombre de diamants par couleur - Altair',
```

```
width=500, height=350
)
chart.show()
```



```
In [57]: clarity_counts = diamonds_df['clarity'].value_counts().reset_index()
clarity_counts.columns = ['clarity', 'count']
```

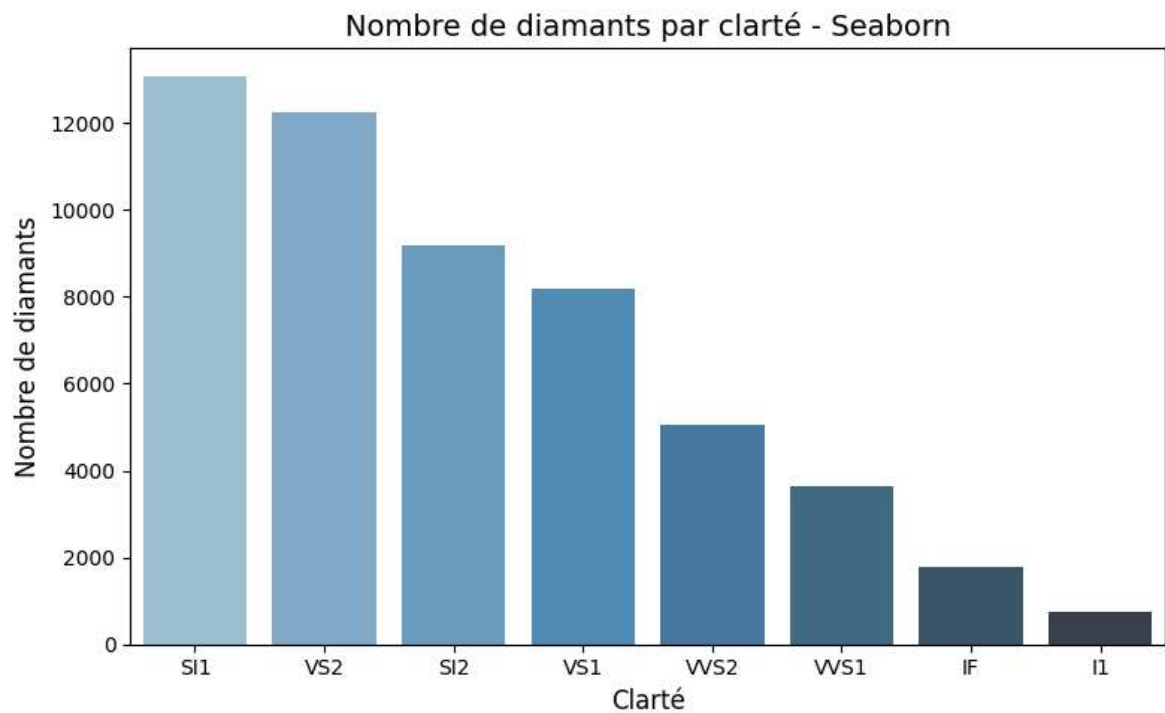
```
In [58]: # -- Matplotlib --
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(clarity_counts['clarity'], clarity_counts['count'], color='steelblue', ed
ax.set_xlabel('Clarté', fontsize=12)
ax.set_ylabel('Nombre de diamants', fontsize=12)
ax.set_title('Nombre de diamants par clarté - Matplotlib', fontsize=14)
plt.tight_layout()
plt.show()
```



```
In [59]: # -- Seaborn --
plt.figure(figsize=(8, 5))
sns.countplot(data=diamonds_df, x='clarity', order=clarity_counts['clarity'], palette='magma')
plt.xlabel('Clarté', fontsize=12)
plt.ylabel('Nombre de diamants', fontsize=12)
plt.title('Nombre de diamants par clarté - Seaborn', fontsize=14)
plt.tight_layout()
plt.show()
```

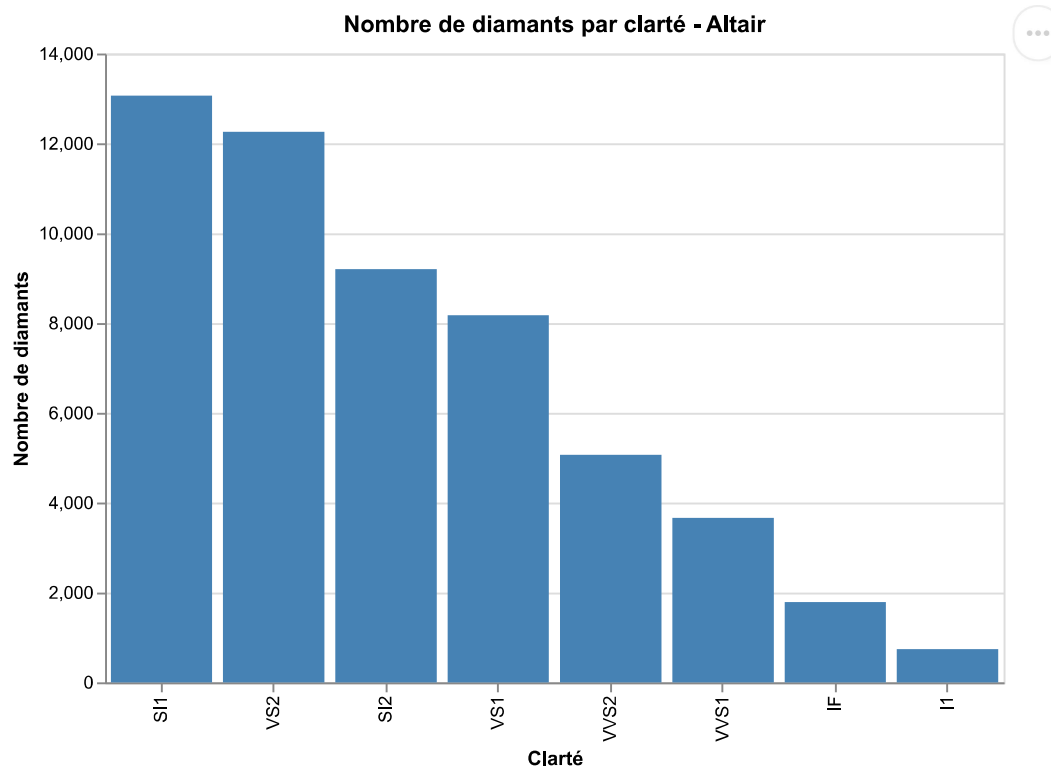
C:\Users\thoma\AppData\Local\Temp\ipykernel_127688\1061087133.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



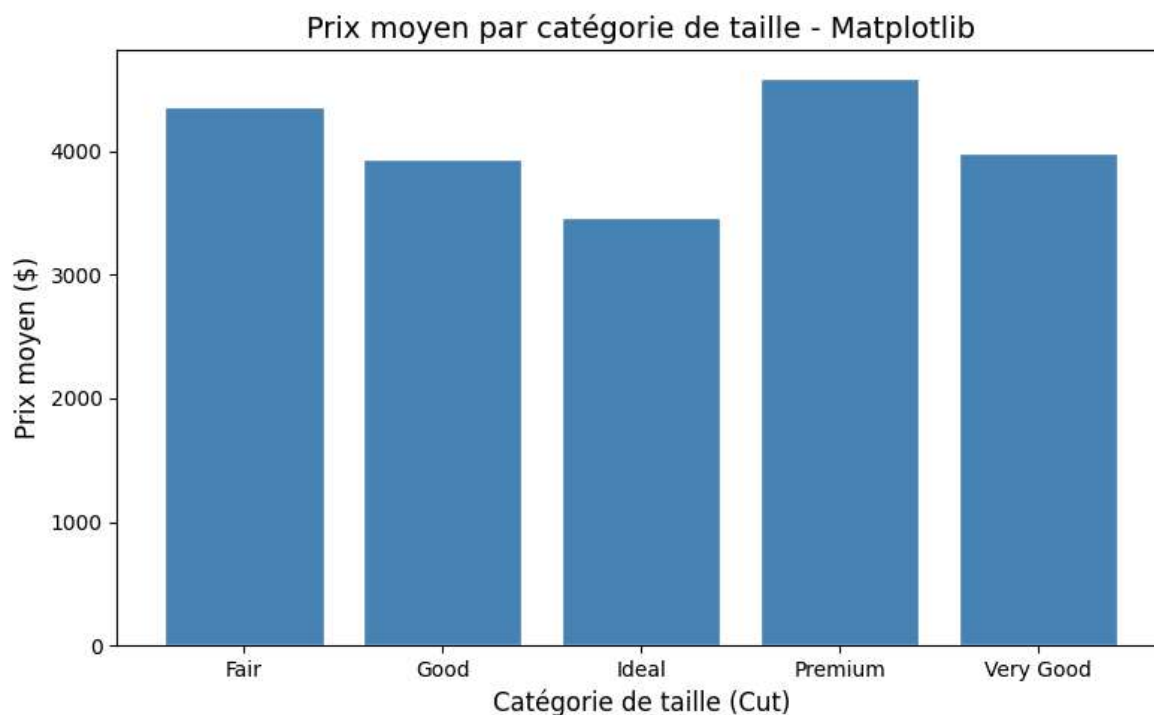
```
In [60]: # -- Plotly --
fig = px.bar(
    clarity_counts,
    x='clarity', y='count',
    title='Nombre de diamants par clarté - Plotly',
    labels={'clarity': 'Clarté', 'count': 'Nombre de diamants'},
    color='count', color_continuous_scale='Blues'
)
fig.update_layout(coloraxis_showscale=False)
fig.show()
```

```
In [61]: # -- Altair --
chart = alt.Chart(diamonds_df).mark_bar(color='steelblue').encode(
    x=alt.X('clarity:N', title='Clarté', sort='-y'),
    y=alt.Y('count():Q', title='Nombre de diamants'),
    tooltip=['clarity:N', 'count():Q']
).properties(
    title='Nombre de diamants par clarté - Altair',
    width=500, height=350
)
chart.show()
```



```
In [62]: avg_price_cut = diamonds_df.groupby('cut')['price'].mean().round(2).reset_index(
avg_price_cut.columns = ['cut', 'avg_price']
```

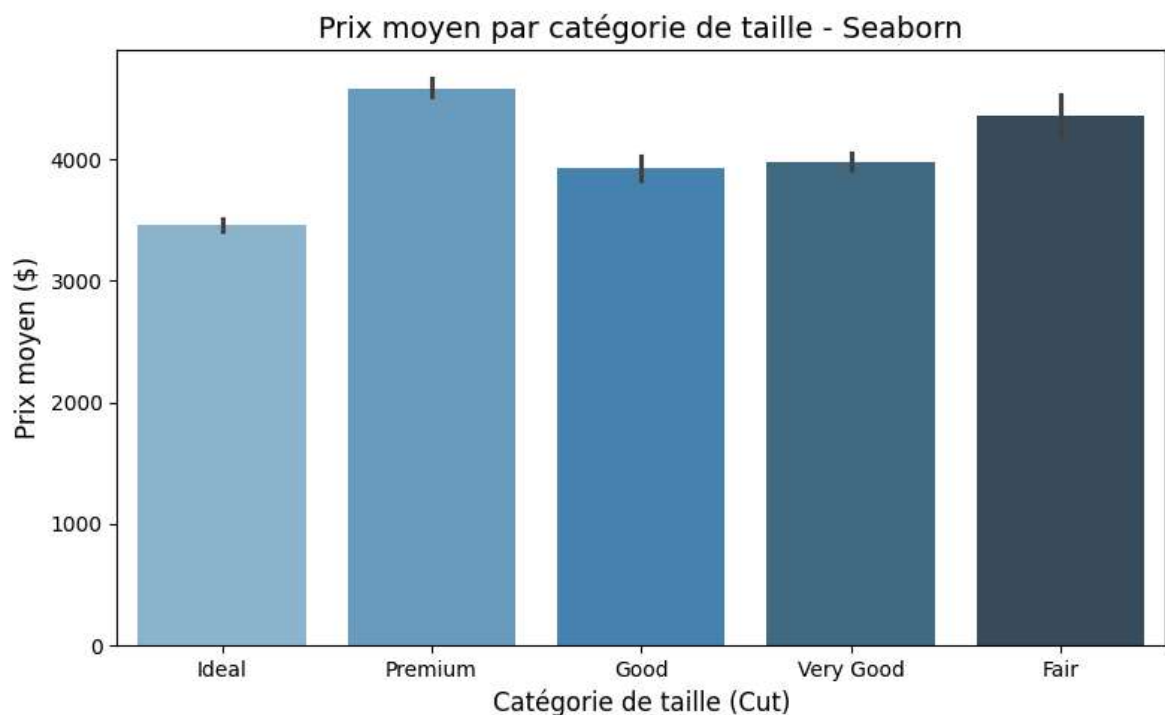
```
In [63]: # -- Matplotlib --
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(avg_price_cut['cut'], avg_price_cut['avg_price'], color='steelblue', edge
ax.set_xlabel('Catégorie de taille (Cut)', fontsize=12)
ax.set_ylabel('Prix moyen ($)', fontsize=12)
ax.set_title('Prix moyen par catégorie de taille - Matplotlib', fontsize=14)
plt.tight_layout()
plt.show()
```



```
In [64]: # -- Seaborn -- (calcule la moyenne directement, sans groupby())
plt.figure(figsize=(8, 5))
sns.barplot(data=diamonds_df, x='cut', y='price', palette='Blues_d', estimator='mean')
plt.xlabel('Catégorie de taille (Cut)', fontsize=12)
plt.ylabel('Prix moyen ($)', fontsize=12)
plt.title('Prix moyen par catégorie de taille - Seaborn', fontsize=14)
plt.tight_layout()
plt.show()
```

C:\Users\thoma\AppData\Local\Temp\ipykernel_127688\872603952.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



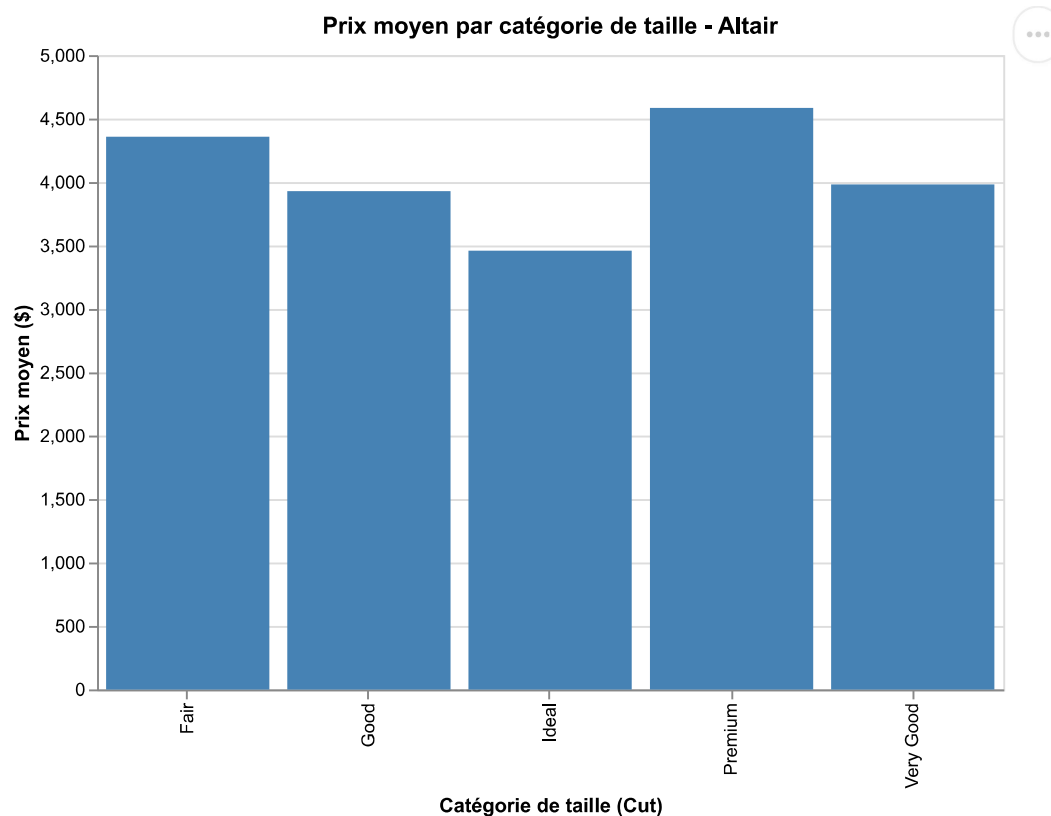
```
In [65]: # -- Plotly --
fig = px.bar(
    avg_price_cut,
    x='cut', y='avg_price',
    title='Prix moyen par catégorie de taille - Plotly',
    labels={'cut': 'Catégorie de taille', 'avg_price': 'Prix moyen ($)'},
    color='avg_price', color_continuous_scale='Blues',
    text='avg_price'
)
fig.update_traces(texttemplate='%{text:.0f}$', textposition='outside')
fig.update_layout(coloraxis_showscale=False)
fig.show()
```

```
In [66]: # -- Altair -- (calcule la moyenne directement via mean(price))
chart = alt.Chart(diamonds_df).mark_bar(color='steelblue').encode(
    x=alt.X('cut:N', title='Catégorie de taille (Cut)'),
    y=alt.Y('mean(price):Q', title='Prix moyen ($)'),
    tooltip=['cut:N', 'mean(price):Q']
)
```

```

).properties(
    title='Prix moyen par catégorie de taille - Altair',
    width=500, height=350
)
chart.show()

```



```

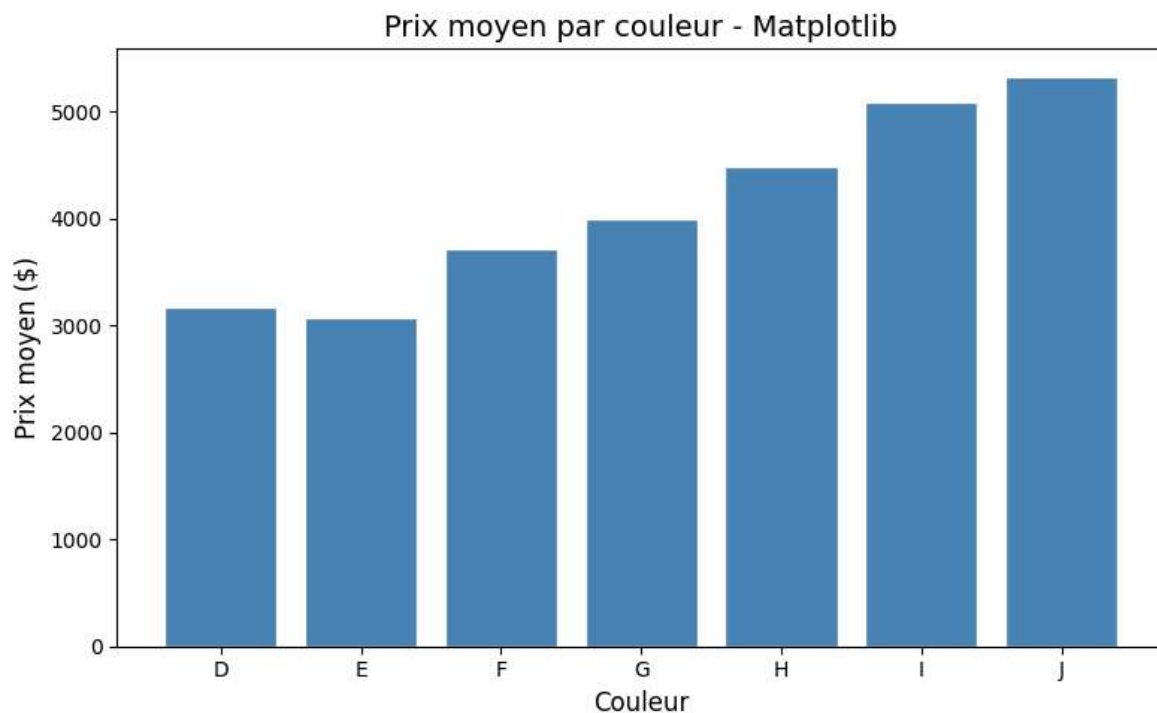
In [67]: avg_price_color = diamonds_df.groupby('color')['price'].mean().round(2).reset_index()
avg_price_color.columns = ['color', 'avg_price']

```

```

In [68]: # -- Matplotlib --
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(avg_price_color['color'], avg_price_color['avg_price'], color='steelblue')
ax.set_xlabel('Couleur', fontsize=12)
ax.set_ylabel('Prix moyen ($)', fontsize=12)
ax.set_title('Prix moyen par couleur - Matplotlib', fontsize=14)
plt.tight_layout()
plt.show()

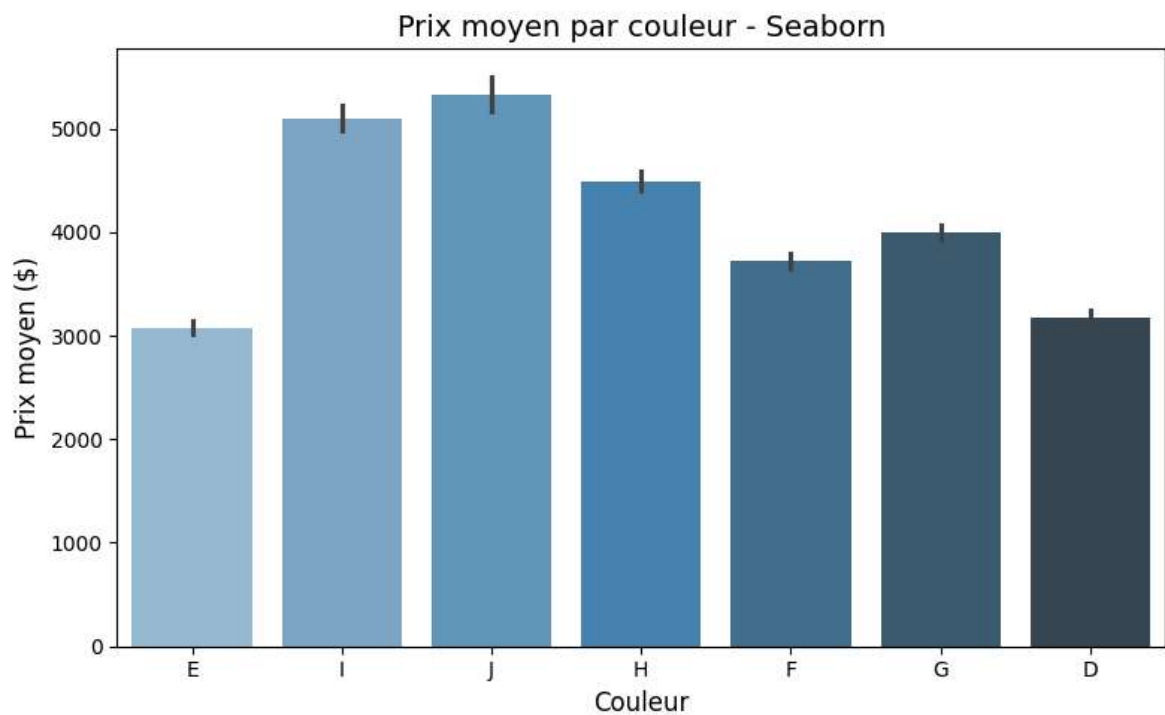
```



```
In [69]: # -- Seaborn --
plt.figure(figsize=(8, 5))
sns.barplot(data=diamonds_df, x='color', y='price', palette='Blues_d', estimator=
plt.xlabel('Couleur', fontsize=12)
plt.ylabel('Prix moyen ($)', fontsize=12)
plt.title('Prix moyen par couleur - Seaborn', fontsize=14)
plt.tight_layout()
plt.show()
```

C:\Users\thoma\AppData\Local\Temp\ipykernel_127688\4151262872.py:3: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.



```
In [70]: # -- Plotly --
fig = px.bar(
    avg_price_color,
    x='color', y='avg_price',
    title='Prix moyen par couleur - Plotly',
    labels={'color': 'Couleur', 'avg_price': 'Prix moyen ($)'},
    color='avg_price', color_continuous_scale='Blues',
    text='avg_price'
)
fig.update_traces(texttemplate='%{text:.0f}$', textposition='outside')
fig.update_layout(coloraxis_showscale=False)
fig.show()
```

```
In [71]: # -- Altair --
chart = alt.Chart(diamonds_df).mark_bar(color='steelblue').encode(
    x=alt.X('color:N', title='Couleur'),
    y=alt.Y('mean(price):Q', title='Prix moyen ($)'),
    tooltip=['color:N', 'mean(price):Q']
).properties(
    title='Prix moyen par couleur - Altair',
    width=500, height=350
)
chart.show()
```

